数据库原理

Database Principle

陶宏才，李晓航

西南交通大学

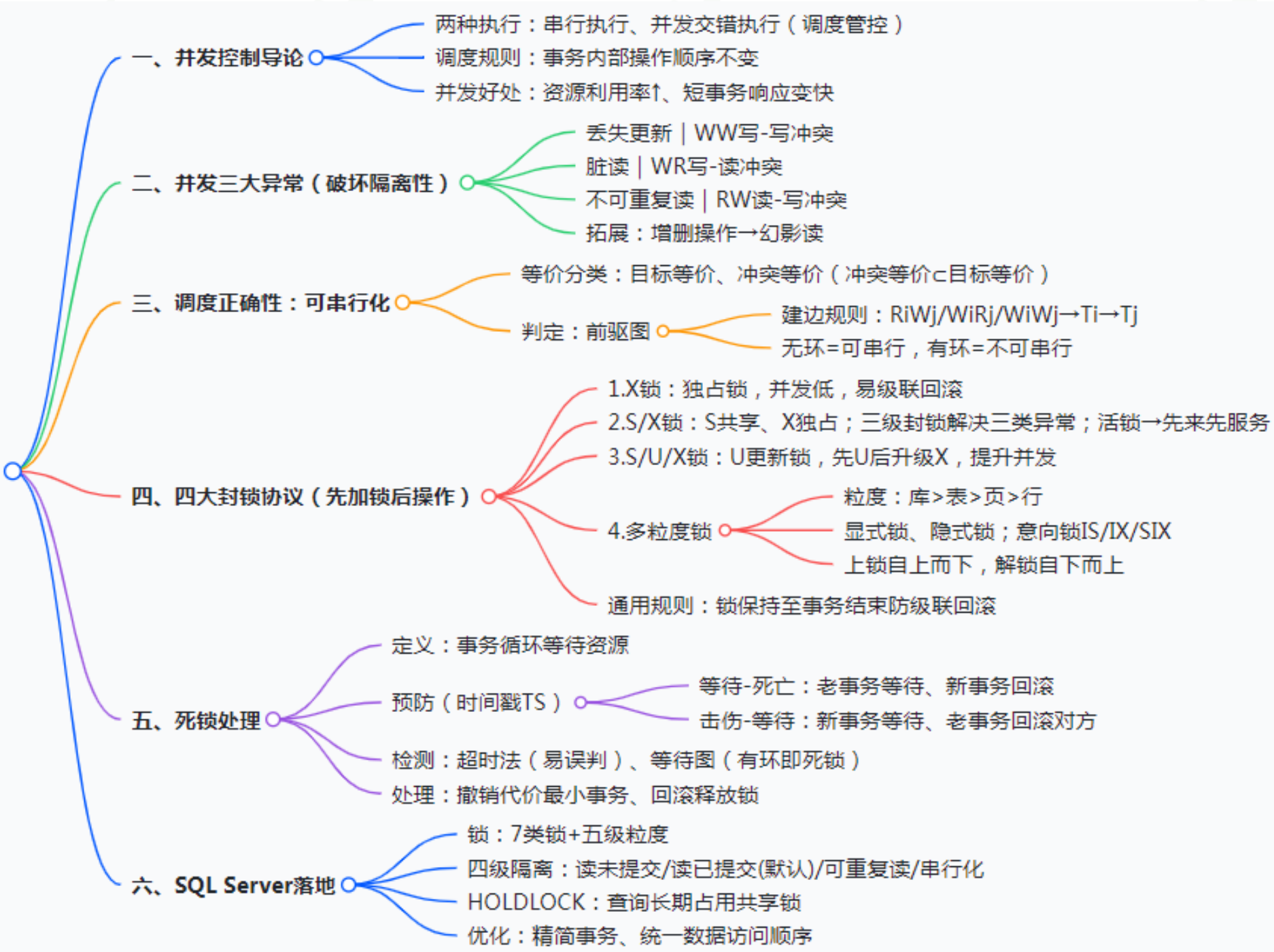
计算机与人工智能学院

第8章 并发控制

学习目的和要求

- 为何要并发？
- 并发执行可能引起哪些问题？
- 如何知道实行并发后结果是正确的？
- 并发执行如何控制实现？
- 死锁的检测、处理和防止
- SQL SERVER中的锁与加锁



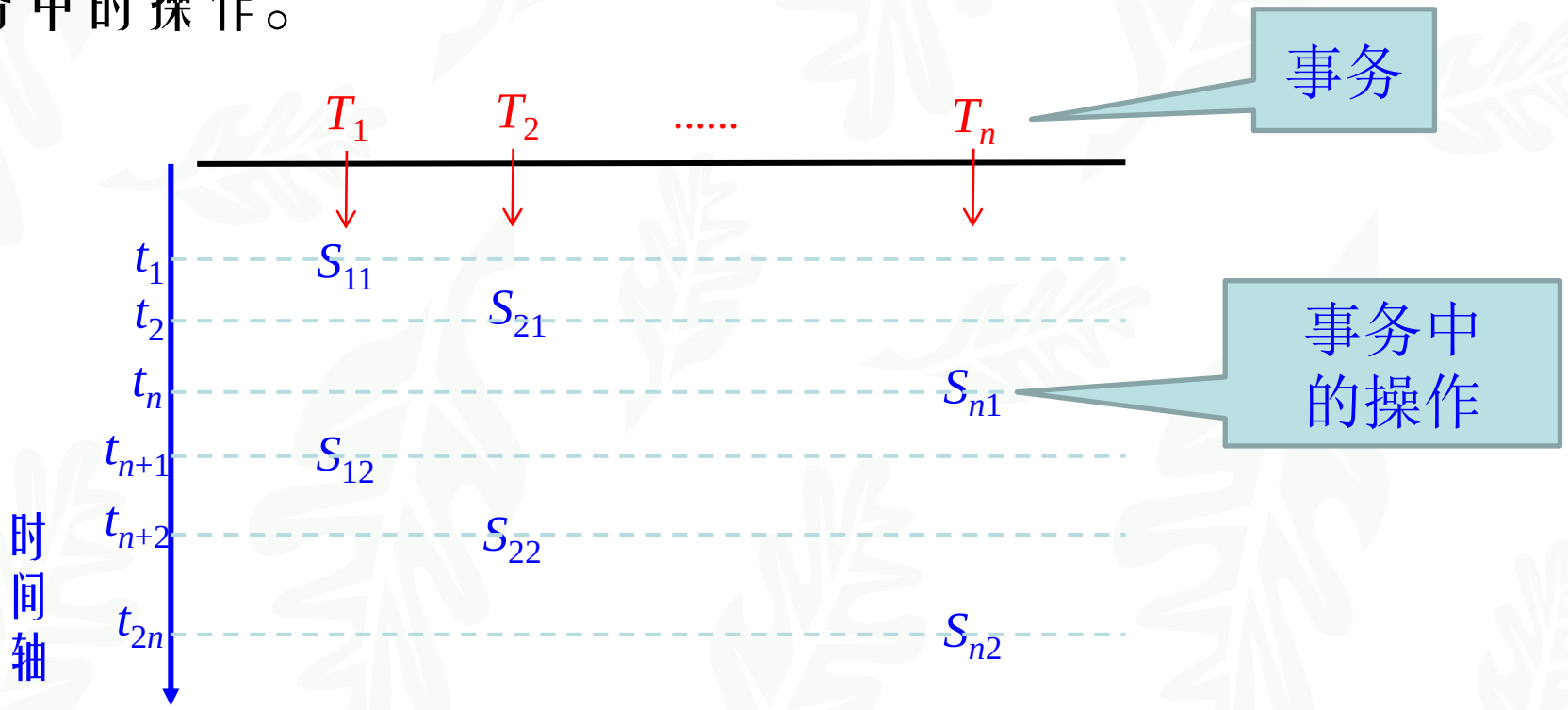

一、并发控制导论
两种执行：串行执行、并发交错执行（调度管控）
调度规则：事务内部操作顺序不变
并发好处：资源利用率↑、短事务响应变快
二、并发三大异常（破坏隔离性）
丢失更新｜WW写-写冲突
脏读｜WR写-读冲突
不可重复读｜RW读-写冲突
拓展：增删操作→幻影读
三、调度正确性：可串行化
等价分类：目标等价、冲突等价（冲突等价⊂目标等价）
判定：前驱图
建边规则：RiWj/WiRj/WiWj→Ti→Tj
无环=可串行，有环=不可串行
四、四大封锁协议（先加锁后操作）
1.X锁：独占锁，并发低，易级联回滚
2.S/X锁：S共享、X独占；三级封锁解决三类异常；活锁→先来先服务
3.S/U/X锁：U更新锁，先U后升级X，提升并发
4.多粒度锁
粒度：库>表>页>行
显式锁、隐式锁；意向锁IS/IX/SIX
上锁自上而下，解锁自下而上
通用规则：锁保持至事务结束防级联回滚
五、死锁处理
定义：事务循环等待资源
预防（时间戳TS）
等待-死亡：老事务等待、新事务回滚
击伤-等待：新事务等待、老事务回滚对方
检测：超时法（易误判）、等待图（有环即死锁）
处理：撤销代价最小事务、回滚释放锁
六、SQL Server落地
锁：7类锁+五级粒度
四级隔离：读未提交/读已提交(默认)/可重复读/串行化
HOLDLOCK：查询长期占用共享锁
优化：精简事务、统一数据访问顺序

8.1 并发控制导论

一.数据库系统中的并发

事务串行访问(Serial Access)：事务顺序执行，一个事务完全结束后才开始另一事务。

并发访问(Concurrent Access)：多个事务同时启动，但交错执行各事务中的操作。



操作的交错执行：需要**调度**（Schedule）。

补充：事务执行方式（引自《数据库系统概论》（第6版））

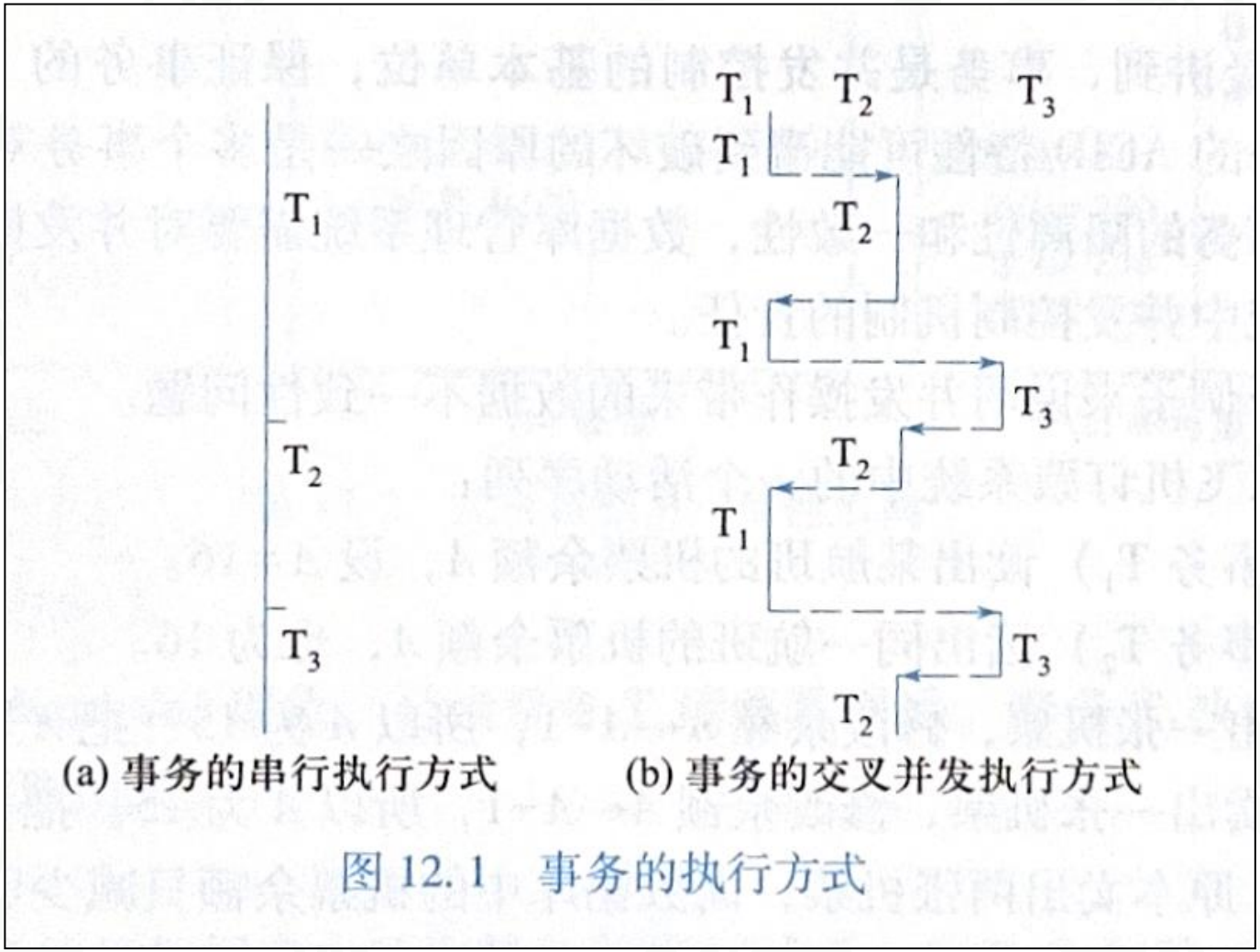


图 12.1　事务的执行方式

在单处理器系统中，事务的并发执行实际上是这些并发事务的操作轮流交替运行，并没有真正的并发运行；但并发执行可以提高系统的资源利用率，改善短事务的响应时间。

二.调度概念

调度：“事务集”中的一串有序操作集。

对调度的要求：一个事务中操作在调度中的顺序应该与它们在事务中的顺序一致。

每个事务最后的行动：COMMIT和ROLLBACK。

T1	T2
R(A)	
W(A)	
	R(B)
	W(B)
	Commit
R(C)	
W(C)	
Commit	

涉及两个事务的并发调度

该并发调度的表示：

$$S = R_{T1}(A)W_{T1}(A)R_{T2}(B)W_{T2}(B)R_{T1}(C)W_{T1}(C)$$

注：如果对这两个事务串行调度，则可能的调度是：

$$S = R_{T1}(A)W_{T1}(A)\ R_{T1}(C)W_{T1}(C)\ R_{T2}(B)W_{T2}(B)$$ 或

$$S = R_{T2}(B)W_{T2}(B)R_{T1}(A)W_{T1}(A)\ R_{T1}(C)W_{T1}(C)$$

三.并发的目的（与串行执行比较）

可回答第一个问题。

(1) 改善系统的资源利用率；

(2) 改善短事务的响应时间。



注：在单处理器系统中，事务的并发执行实际上是这些并发事务的操作轮流交替运行，并没有真正的并发运行；而多处理器系统才能实现多个事务的操作真正的并发运行。

本章讨论的并发控制是以单处理器系统为基础的。

8.2 并发执行可能引起的问题

只考虑R/W(Select/Update)操作，事务如不加控制地并发执行，会产生如下三个问题（**都破坏了事务的隔离性**）。

一. 丢失更新(Lost Update)



(a) 事务调度　　(b) 按(a)调度执行的结果

图 丢失更新

亦称：覆盖未提交的数据(Overwriting Uncommitted Data)。

原因：因两个事务对同一数据并发地写入引起，称写-写冲突(write–write conflict)。

结果：与串行地执行两个事务的结果不一致。

二. 读脏数据(Dirty Read)

事务 T1	事务 T2
R(A)	
W(A)	
	R(A)
RollBack	
	…

操作执行次序

(a) 事务调度

事务 T1	事务 T2
R(A)：5	
W(A)：6→A	
	R(A)：6
RollBack： A 值恢复为 5	
	但 T2 仍可能用 6 这个“脏”数据作为 A 的值做其他事情

(b) 按(a)调度执行的结果

图 读“脏”数据

亦称：读未提交的数据(Reading Uncommitted Data)。

原因：由于一事务读另一事务尚未提交的数据引起，称为写-读冲突(write-read conflict)。

结果：有可能读到要回退的数据。

三.读值不可复现(Unrepeatable Read)



(a) 事务调度

事务 T1	事务 T2
R(A)：5	
	W(A)：6→A
R(A)：6 ？	

(b) 按(a)调度执行的结果

图　读值不可复现

原因：该问题因读-写冲突(read-write conflict)引起。

结果：第二次读的值与前次读的值不同。

注：如再考虑Insert/Delete操作，则还可能出现幻影(Phantom)问题。

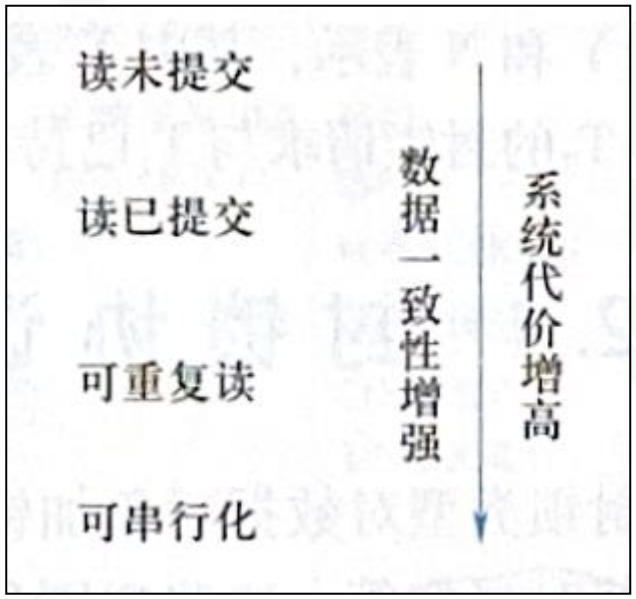
结论1：由上可以看出，把不同事务中对同一对象进行WW、RW或WR的操作对进行交错都会造成冲突。因此，在安排事务操作交错（即并发执行）时，应将这些操作对隔离。

结论2：并发操作产生数据不一致的主要原因是并发操作破坏了事务的“隔离性”。并发控制机制就是要用正确的方式调度并发操作，使一个用户事务的执行不受其他事务的干扰，从而避免数据不一致。

补充：SQL标准中的四种事务隔离级别（引自《数据库系统概论》（第6版））

表 12.1 事务隔离级别与数据不一致性的关系

事务隔离级别	数据不一致性			
	丢失修改	脏读	不可重复读	*幻读
读未提交	否	是	是	是
读已提交	否	否	是	是
可重复读	否	否	否	是
可串行化	否	否	否	否



目前大多数数据库（如SQL Server、Oracle等）默认的事务隔离级别是“读已提交”，而MySQL是“可重复读”

8.3 并发执行的正确性准则

本节内容回答第三个问题，即：如何知道或判定并发执行后结果是否正确呢？

一.基本概念

并发原则：既要交错执行，以充分利用系统资源、加快短事务响应时间；又要避免访问冲突。

并发事务调度(Schedule)：是一串事务中所有操作的顺序序列。

事务调度原则：调度中，不同事务的操作可以交错，但需保持各个事务内操作的次序。

约定：为便于问题描述，假定DB对象*O*总被读入程序的*O*变量，将事务T中读对象*O*的操作表示为$R_T(O)$，写表示成$W_T(O)$，或用事务号作下标。

并发调度描述：

$$S=\ldots.R_{T1}(x)\ldots.W_{T2}(x)\ldots..R_{T1}(y)\ldots.$$

二.等价调度

问题提出：对同一事务集，可能有多种调度。

调度等价：如有两个调度S1和S2，在DB的任一初始状态下，所有读出的数据都是一样的，留给DB的最终状态也是一样的，则称S1和S2是等价（Equivalence）的。

目标等价(View Equivalence)：以上的调度等价定义是一个普遍定义，亦称为目标等价。

考虑冲突操作：不同事务的一对操作，有些是冲突的(指不同事务对同一个数据的读写和写写操作)，有些是不冲突的。

冲突等价（Conflict Equivalence）：凡是通过调换S中不冲突操作所得的新调度，称S的冲突等价调度。

结论：如两个调度是冲突等价的，则一定是目标等价的；反之不亦然。

三.可串行化（Serializability）

概念：对一事务集，如一个并发调度与一个串行调度等价，则称此并发调度是可串行化的(Serializable)。

可串行化分类：对应两个调度等价，可串行化亦分：目标可串行化和冲突可串行化。

推论1：因冲突等价一定是目标等价，故冲突可串行化调度也一定是目标可串行化调度。

基本论点：串行调度中，各事务操作没有交错，即不会相互干扰，故不会产生并发引起的问题。

推论2：可串行化调度与某一串行调度等价，故也不会产生并发所引起的问题。

并发执行的正确性准则：在当前的DBMS中，均以“冲突可串行化”作为并发执行的正确性准则。

注意：

① 对n个事务，可有$n!$种排列，即有$n!$种串行调度。每个串行调度执行的结果可能不一样，可串行化准则只要求并发调度和其中某一个串行调度等价即可。

② 不同的可串行化调度是不一定等价的，即n个事务交付系统执行后，由于调度不同，可能产生不同的结果，这是允许的。除非特别规定，用户对这n个事务的执行顺序无要求。



四.调度可串行化的测试

方法：一个调度是否可串行化，可用其前趋图(Precedence Graph)来测试。

前趋图：是一有向图G=(V, E)，V(Vertex)为顶点集合，E(Edge)为边集合。在此，V包含所有参与调度的事务，而边可通过分析冲突操作来决定。

前趋图构造：如下列条件之一成立，则在E中可加一条边$T_i \rightarrow T_j$。

① $R_i(x)$在$W_j(x)$之前；

② $W_i(x)$在$R_j(x)$之前；

③ $W_i(x)$在$W_j(x)$之前。

注意：R和W均针对同一个数据X

是否可串行化测试方法：如前趋图中有回路，则S不可能等价于任何串行调度；如前趋图无回路，则可找到S的一个等价的串行调度。

寻找等价串行调度的方法：因图中无回路，必有一入度为零的结点，将这些结点及其边从图中移去存放于一个队列中（次序任意），对所剩图作同样处理。不过，移去的结点要放在队列中已存结点的后面，如此继续下去，直到所有结点移入队列中。按队中结点次序串行排列各事务，即可得到一个等价的串行调度。

示例：事务集={T1，T2，T3，T4}

$$S=W_{T3}(y)R_{T1}(x)R_{T2}(y)W_{T3}(x)W_{T2}(x)W_{T3}(z)R_{T4}(z)W_{T4}(x)$$

问题：

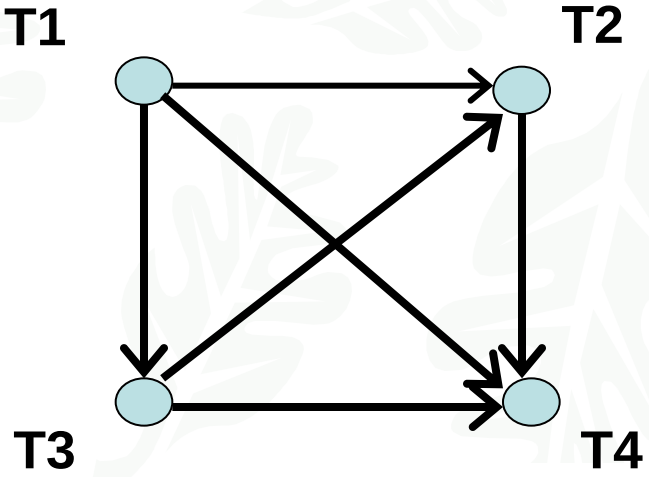
① 该调度是否可串行化?

② 如是，与其等价的串行调度是什么?

$S=W_{T3}(y)R_{T1}(x)R_{T2}(y)W_{T3}(x)W_{T2}(x)W_{T3}(z)R_{T4}(z)W_{T4}(x)$

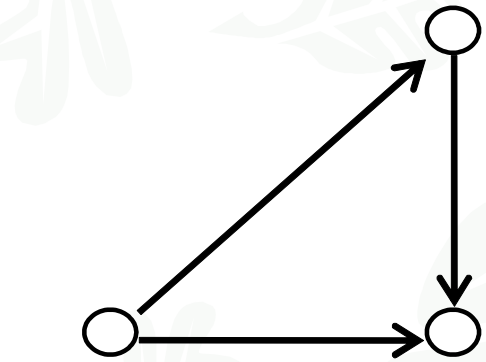
问题1答案：

图中无回路，因此，该并发调度为可串行调度！



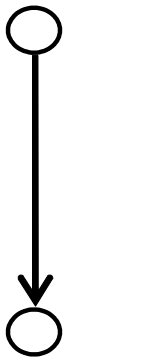
问题2答案：

与该并发调度等价的串行调度为：T1、T3、T2、T4。



队列：

T1



队列：

T1, T3



队列：

T1, T3, T2

队列：

T1, T3, T2, T4

8.4 基于锁的并发控制协议

一. 概述

问题提出：DBMS并发控制的任务是要保证事务执行的可串行化。但用前面的测试方法来保证调度可串行化是不现实的：一是不可能事先定好调度，二是事务集随机动态、不固定。那么，DBMS如何实现事务的并发控制呢?

并发控制通常的实现方法：由DBMS定好一个协议，按此协议执行事务，即可保证可串行化，而不必关心具体的调度。

加锁协议(Locking Protocol)：是这类协议中的一种（其它还有：基于时间戳的并发控制、多版本并发控制MVCC），它是用加锁来实现并发控制，即是在操作前对操作对象加锁——先加锁后操作。

定理：事务只要遵守加锁协议，即可保证调度的可串行化。

加锁协议类型：

① X锁　　② (S, X) 锁

③ (S, U, X) 锁　　④ 多粒度加锁或封锁

二. X锁加锁协议

X锁协议：该协议中，只有一种锁，即X（eXclusive）锁，亦称排他/独占锁，既可用于读操作，亦可用于写操作。一个事务对某个数据对象加排他锁后，其它事务就不能对该数据对象加锁。

相容（或冲突）矩阵：加锁协议可用相容矩阵(compatibility matrix)描述。所谓相容矩阵，是说明锁请求在什么情况下可获准或被拒绝。

X锁协议的相容（或冲突）矩阵：

对象的锁状态	NL	X
加锁申请 X	Y	N

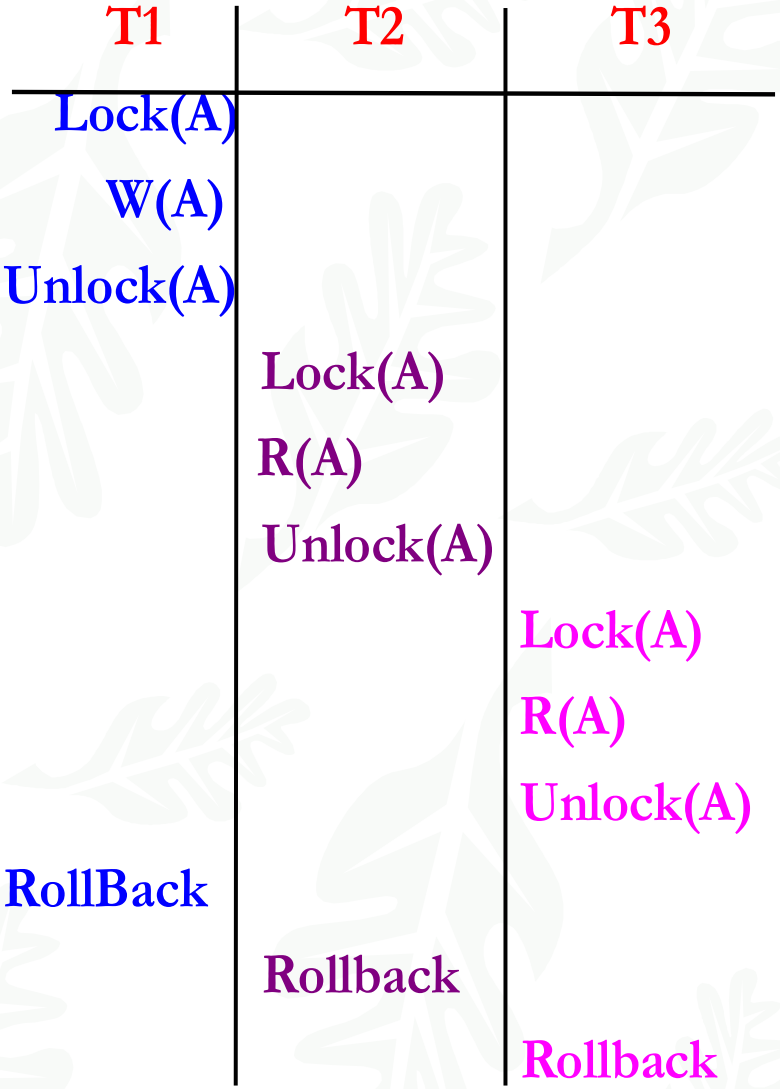
X 锁相容矩阵

说明：① 锁是否相容，实际上就是看对同一数据对象的加锁是否存在有冲突；② Y表示相容，N表示冲突。

级联回退(Cascading Rollback)现象：

级联回退原因：过早释放锁。

加锁协议补丁(Patch)1：为避免级联回退，要求不管是写操作锁还是读操作锁都应保持到事务结束(End of Transaction, EOT)才释放。该补丁对其后的加锁协议也适用。



级联回退

两阶段加锁协议（2PL—two phase locking protocol）

一个事务如果其加锁都在所有的锁释放之前，则此事务为两阶段事务。遵守这种加锁限制的协议，称为两阶段加锁协议。

说明：

（1）两阶段事务中的两个阶段，分别是加锁阶段和解锁阶段。加锁阶段，是其所拥有的锁逐步增长的阶段(growing phase)；而解锁阶段，是其所拥有的锁逐步缩减的阶段(shrinking phase)。

（2）两阶段事务中，一旦开始解锁，就不能再对任何数据对象加锁。

（3）两阶段加锁协议不是一个具体的协议，但其思想可融入到具体的加锁协议之中，如X锁协议以及其他加锁协议。

合式(well formed)事务：遵守先加锁、后操作原则的事务。

定理：如并发执行的所有事务都是合式、两阶段事务，则它们的任何调度都是可串行化的（注：2PL是调度可串行化的充分条件，而非必要条件）。

T1	T2
① SLOCK A、SLOCK B	
② R(A)=50，R(B)=100 sum=A+B=150	XLOCK B → 等待
③ 再次 R (A)=50，R (B)=100 sum=150	等待
④ COMMIT UNLOCK A、UNLOCK B	获得 XLOCK B
	R(B)=100，B=B×2，W(B)=200
	COMMIT UNLOCK B

遵守2PL协议

T1	T2
① SLOCK A	
② R(A)=50 UNLOCK A（中途解锁，违规）	
	XLOCK B，R(B)=100，B=B×2，W(B)=200
③ SLOCK B（解锁后又新加锁，破坏 2PL）	COMMIT、UNLOCK B
R(B)=200，sum=50+200=250 UNLOCK B	
④ COMMIT	

不遵守2PL协议

三.（S，X）加锁协议

（S，X）协议：该加锁协议中，有两把锁，即：①S锁(Sharing locks)：用于读操作；②X锁(eXclusive locks)：用于写操作。

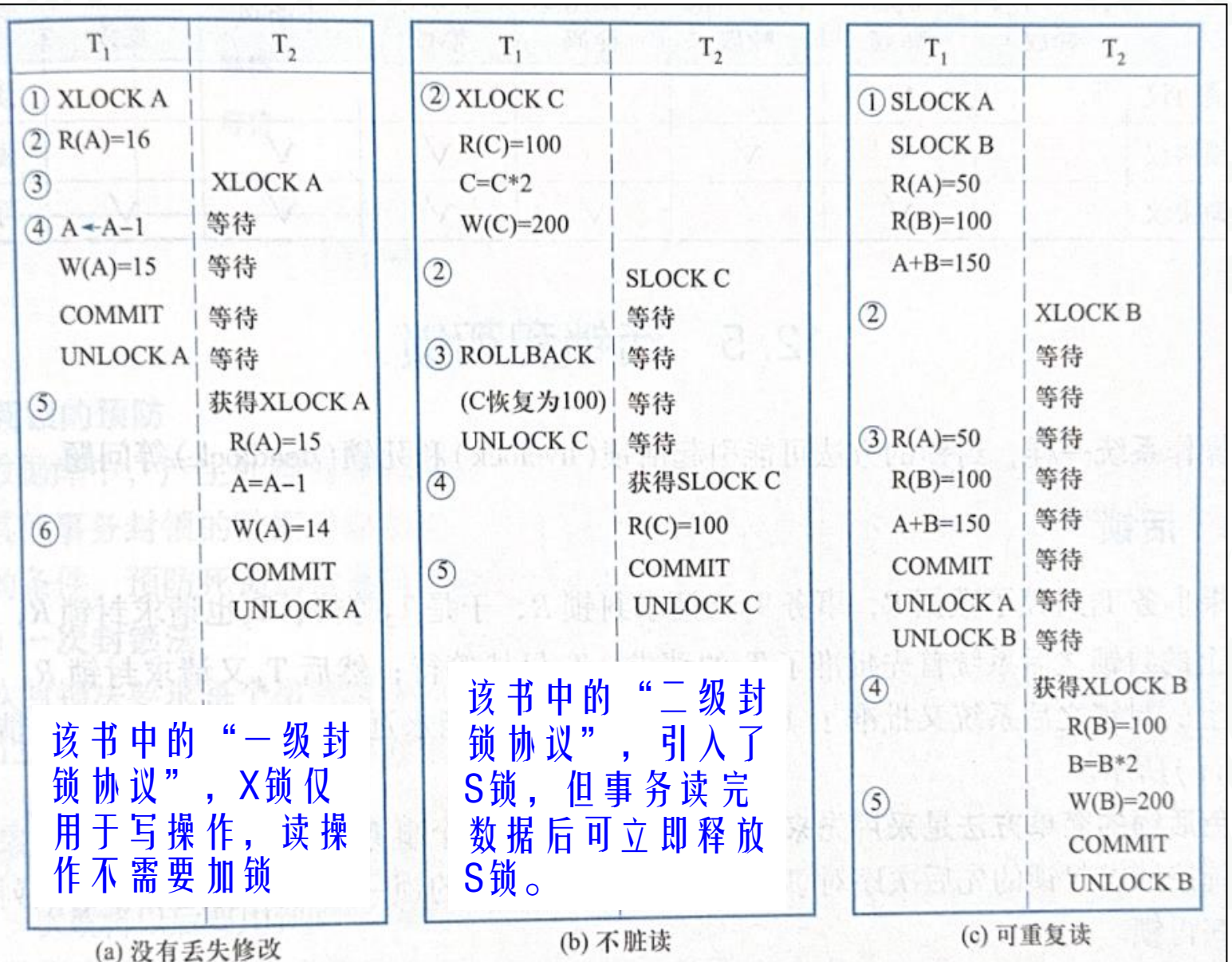
（S，X）协议的相容矩阵：

加锁申请	数据对象的锁状态 NL	S	X
S	Y	Y	N
X	Y	N	N

（S，X）加锁协议相容矩阵

与X锁协议比较：由于S锁只用于读，故S锁与S锁是相容的，即：同一数据对象可允许多个事务并发读，这样比单一的X锁提高了读并发度。

补充：三级封锁协议（引自《数据库系统概论》(第6版)）



T_1	T_2
① XLOCK A	
② R(A)=16	
③	XLOCK A
④ A←A-1	等待
W(A)=15	等待
COMMIT	等待
UNLOCK A	等待
⑤	获得XLOCK A
	R(A)=15
	A=A-1
⑥	W(A)=14
	COMMIT
	UNLOCK A

(a) 没有丢失修改

T_1	T_2
② XLOCK C	
R(C)=100	
C=C*2	
W(C)=200	
②	SLOCK C
	等待
③ ROLLBACK	等待
(C恢复为100)	等待
UNLOCK C	等待
④	获得SLOCK C
	R(C)=100
⑤	COMMIT
	UNLOCK C

(b) 不脏读

T_1	T_2
① SLOCK A	
SLOCK B	
R(A)=50	
R(B)=100	
A+B=150	
②	XLOCK B
	等待
	等待
③ R(A)=50	等待
R(B)=100	等待
A+B=150	等待
COMMIT	等待
UNLOCK A	等待
UNLOCK B	等待
④	获得XLOCK B
	R(B)=100
	B=B*2
⑤	W(B)=200
	COMMIT
	UNLOCK B

(c) 可重复读

图 12.5 使用封锁机制解决三种数据不一致性的示例

该书中的“一级封锁协议”，X锁仅用于写操作，读操作不需要加锁

该书中的“二级封锁协议”，引入了S锁，但事务读完数据后可立即释放S锁。

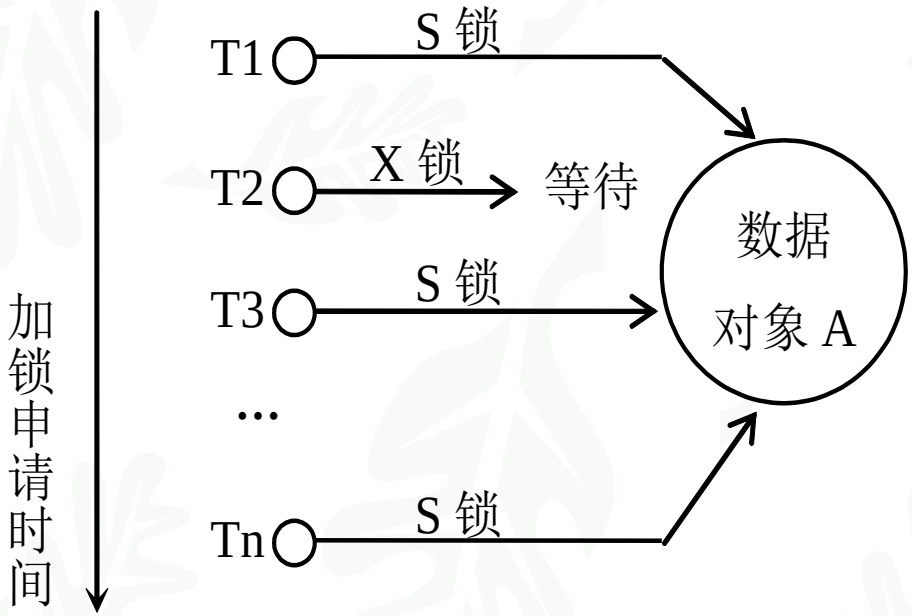
图C是该书的“三级封锁协议”，要求S锁也必须持有到事务结束才能释放，相当于本书加了补丁1的(S, X)协议

补充：三级封锁协议（引自《数据库系统概论》(第6版)）

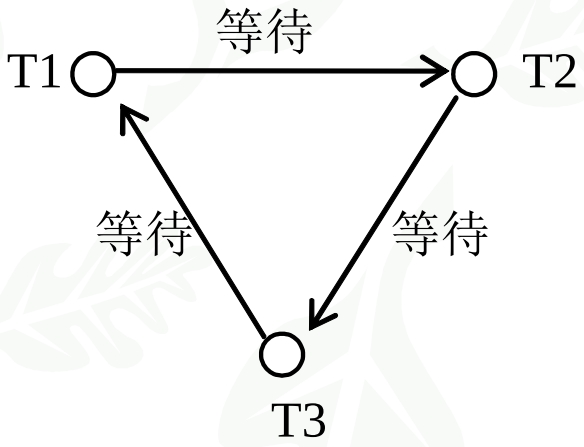
表 12.2 不同级别的封锁协议和一致性保证

封锁协议	X锁		S锁		一致性保证			隔离性级别保证
	操作结束释放	事务结束释放	操作结束释放	事务结束释放	不丢失修改	不脏读	可重复读	
一级封锁协议		√			√			读未提交
二级封锁协议		√	√		√	√		读已提交
三级封锁协议		√		√	√	√	√	可串行化

活锁： 如一数据对象已被加S锁，其他事务要申请对它的X锁，需等待。若此时有其他事务申请对它的S锁，按相容矩阵，应可获准。如不断有事务对它申请S锁，以致该数据对象一直被S锁占据，而X锁的申请迟迟得不到获准，此现象称作活锁（Live Lock）。



注：Ti（i=1,2,…,n）表示第 i 个事务



死锁的循环等待示意图

加锁协议补丁（Patch）2：活锁虽不会导致“死等”，但对系统性能有不良影响。为避免活锁，在加锁协议中应规定“先申请先服务（First Come, First Served）”原则。

说明：为保证可串行化和避免级联回退现象，（S，X）锁一般应保持到EOT才释放。该加锁协议繁简适当，有相当多的DBMS采用此协议。

四.（S，U，X）加锁协议

（S，U，X）协议：该加锁协议中，除S，X两种锁外，又增加了U锁（Update lock）。

增设U锁原因：事务在做更新时，一般分两个步骤：先读后写。在读阶段，先读老内容，在内存中修改后；然后再写入修改后的内容。此过程中，除最后的写入阶段外，为保证需更新的数据对象在读阶段仍可被其它事务访问，而新增此更新锁。

U锁的使用及作用：事务要更新数据对象时，先申请该对象的U锁。数据对象加了U锁后，仍允许其他事务对它加S锁。在最后写入时，事务再申请将U锁升级为X锁。由于不必在事务执行的全过程中加X锁，可进一步提高并发度。

关于补丁1、2的说明：

① 为保证可串行化和避免级联回退等现象，除U锁后来升级为X锁外，S、X锁一般应保持到EOT才释放。

② 数据对象加了U锁后，其他事务可能连续不断地对它加S锁，使U锁迟迟不能升级为X锁。为避免这种活锁，同样也要求遵守“先申请先服务”原则。

（S，U，X）协议的相容矩阵：

加锁申请	数据对象的锁状态			
	NL	S	U	X
S	Y	Y	Y	N
U	Y	Y	N	N
X	Y	N	N	N

（S，U，X）加锁协议相容矩阵

五. 多粒度加锁协议

1. 基本概念

问题提出：在数据库中，一个数据对象的大小可以大到整个数据库、一页、一个表，也可以小到一行(即元组)。对加锁来说，数据对象的大小即是加锁的粒度。

粒度、控制及并发度间关系：加锁单位愈大，加锁起来愈简单，但同时会降低并发度。反之，加锁单位越小，往往需要加很多的锁，所需的控制越复杂，不过并发度可提高。

实际需要：实际应用中，有时要访问大片数据，而有时只访问个别数据，为此，可提供多级加锁单位，根据应用需要加以选用，此即：**多粒度加锁**(Multiple Granularity Locking)，它使得对包含有其它对象的对象加锁更加有效。

实际RDBMS做法：现代大型DBMS一般均支持多粒度加锁，而在微机DBMS中，并发度要求不高，一般以表作为加锁单位，此即：单粒度加锁(Single Granularity Locking)。

RDBMS提供的多粒度锁（从小到大排列）：

① 行级锁，或称行锁（Row Lock）；

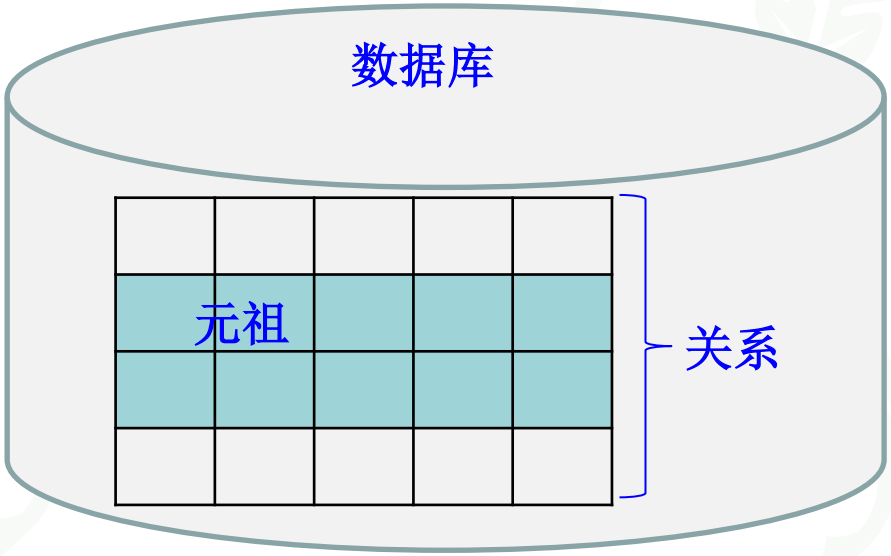
② 页级锁，或称页锁（Page Lock）；

③ 表级锁，或称表锁（Table Lock）。

2. 多粒度加锁的实现

1) 锁冲突检测问题

数据对象的祖先与子孙关系：按数据对象大小（或上下级）关系，区分为祖先（Ancestor）与子孙（Offspring）。



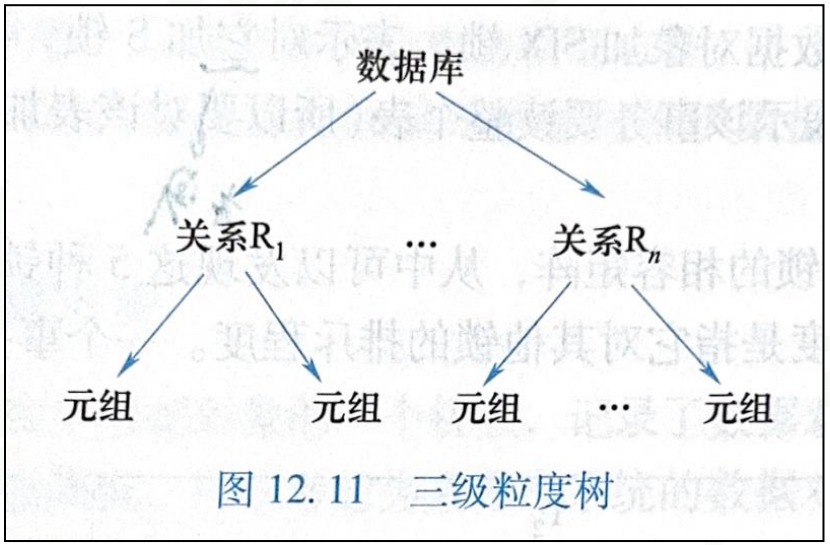
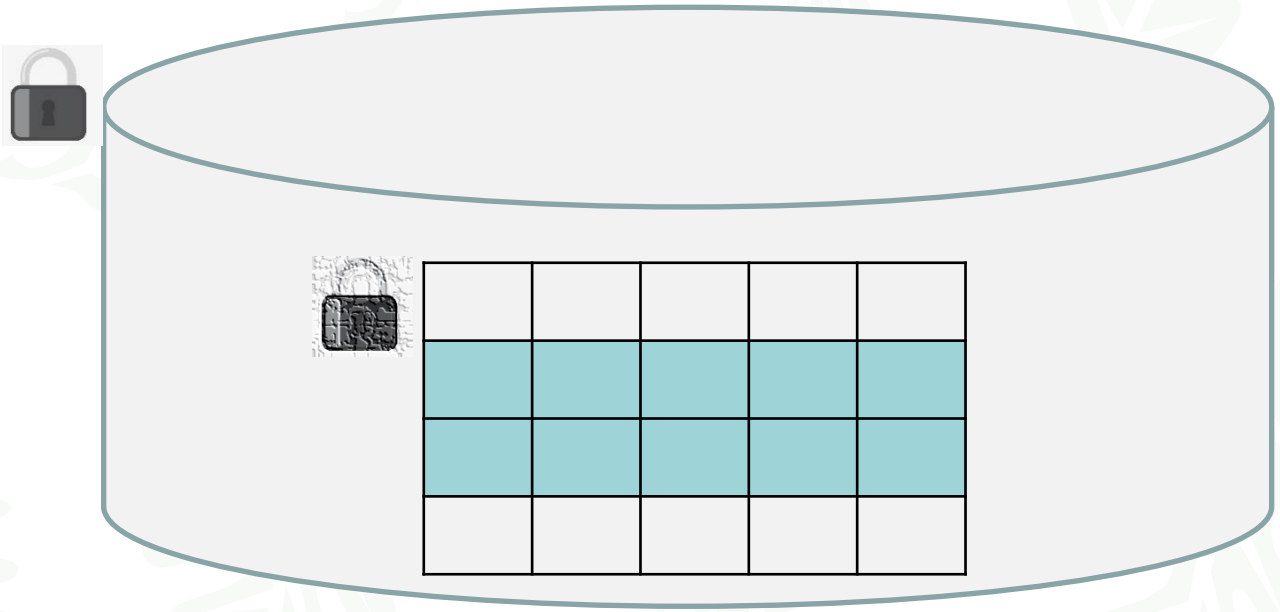


图 12.11　三级粒度树

多粒度加锁下，一个数据对象可能的两种加锁方式：

① 显式加锁(Explicit Locking)：系统应事务的要求，直接对该数据对象的加锁。

② 隐式加锁(Implicit Locking)：该数据对象本身并未被显式加锁，但由于其上级被加锁，故这个数据对象被隐含加锁了。例如，一个表被加锁，则其所有元组和列均隐式被加锁。



说明：尽管区分了显式和隐式加锁，但显式加锁和隐式加锁的效果是一样的。

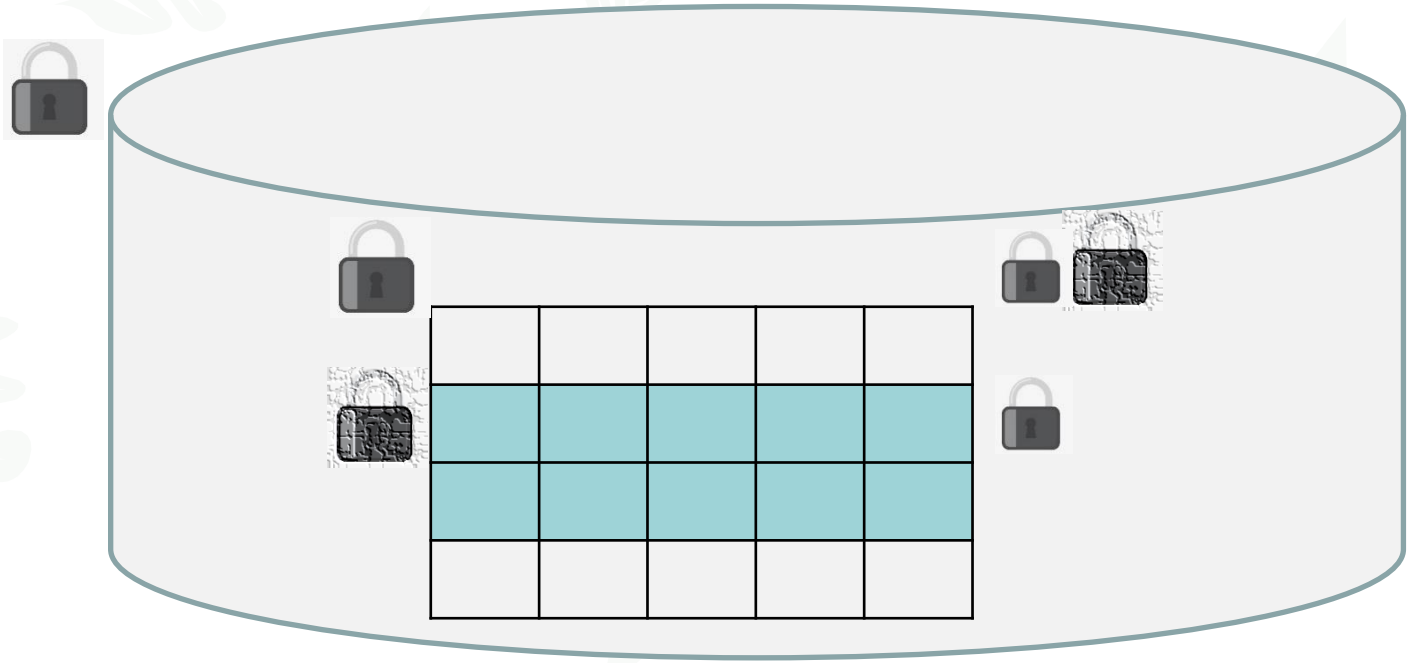
多粒度加锁下的锁冲突检测问题：如只有显式锁，锁的冲突较易发现；若有隐式锁，则检查冲突较复杂。

多粒度加锁下，某事务对某数据对象加锁时的锁冲突检测：

① 检查对象本身，有无显式锁与本事务显式锁冲突；

② 检查其所有祖先，以防本事务的显式锁与其他事务的隐式锁冲突；

③ 检查其所有子孙，以防本事务的隐式锁与其它事务的显式锁相冲突。

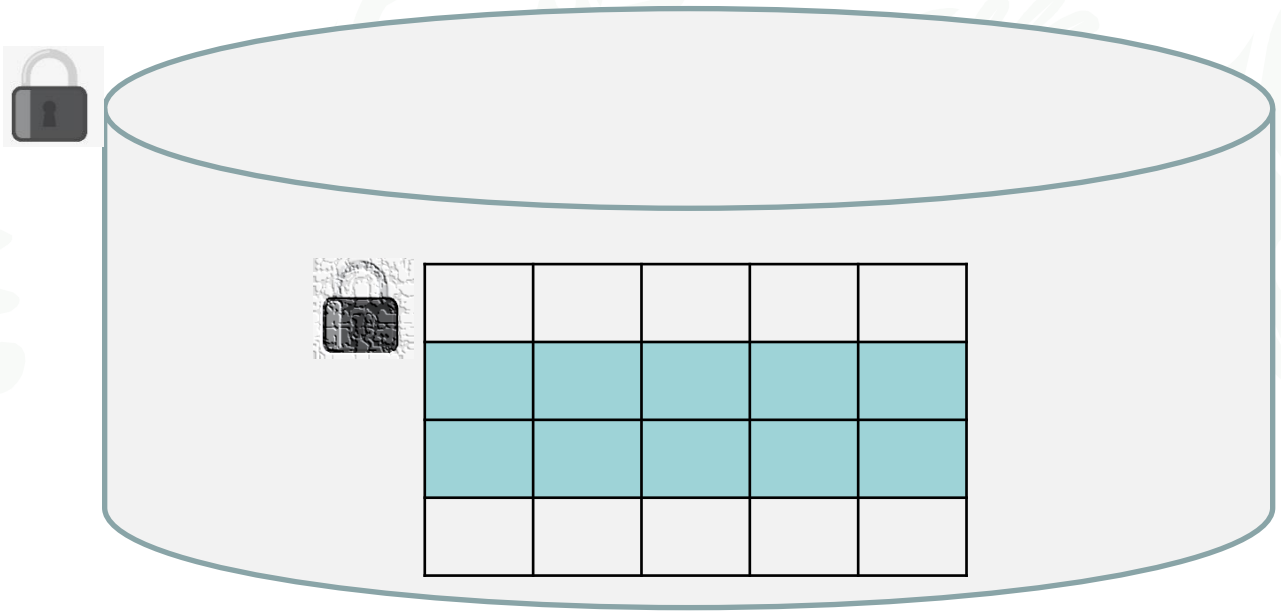


2) 锁冲突检测问题的解决

意向锁的提出：为简化锁冲突的检测，多粒度加锁协议中，除S锁和X锁外，另外引入了三种意向锁(Intent Locks)。该方法在System R首创，并得到广泛应用。

(1) 意向共享IS (Intention Share)锁

加IS锁：一个事务要给一个数据对象加S锁，必须首先将其祖先加IS锁。也就是说，如果一个数据对象被加了IS锁，表示其某些子孙加了或准备加S锁。



锁冲突关系：IS锁与X锁冲突。

锁冲突检测：

① 如果一事务对所操作对象的祖先的IS锁申请成功，就可保证再给该对象加S锁时，不会造成S锁与该对象的隐式锁发生冲突。因为该对象的祖先的锁是本事务加的，当然就不再有这种冲突存在；

② 如事务给数据对象加了S锁，那么其它事务就不能给其任何祖先加X锁，因为其祖先已被该事务加了IS锁。

(2) 意向排他IX (Intention eXclusive)锁

加IX锁：一个事务要给一个数据对象加X锁，必须首先将其祖先加IX锁。也就是说，如果一个数据对象被加了IX锁，表示其某些子孙加了或准备加X锁。

锁冲突关系：IX锁与S、X以及后面的SIX锁冲突。

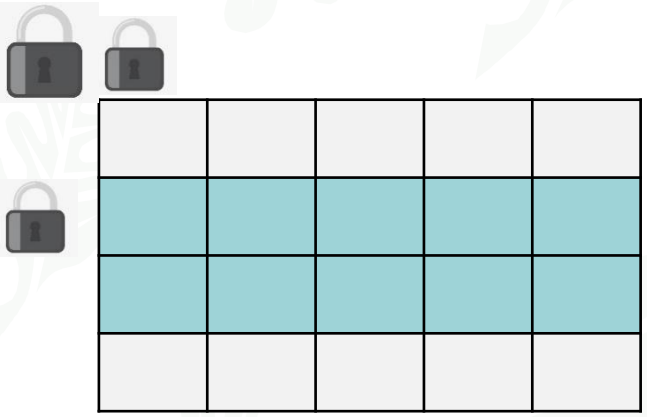
锁冲突检测：

① 如果一事务对所操作对象的祖先的IX锁申请成功，就可保证再给该对象加X锁时，不会造成X锁与该对象的隐式锁发生冲突，因为该对象的祖先的锁是本事务加的，当然就不再有这种冲突存在；

② 如事务给数据对象加了X锁，那么其它事务就不能给其任何祖先加S或X锁，因为其祖先已被该事务加了IX锁，而IX锁与S锁和X锁是冲突的。

(3) 共享意向排他SIX (Share Intention eXclusive)锁

SIX锁的引入或SIX的应用情形：实际应用中，常常有这种情形，即：一个事务需要读整张表，并修改其中个别行（或元组）。也就是说，该事务需要整张表的S锁和IX锁，这样，该事务才能接着以X锁锁住其中要修改的行。然而，由于S锁与IX锁是不相容的，不可能同时给整张表既加S锁又加IX锁。为解决此问题，定义了SIX锁。



SIX锁的等价关系：SIX锁在逻辑上等价于同时拥有S锁和IX锁。

锁冲突关系：SIX锁与那些同S锁或IX锁不相容的锁冲突，即SIX与IX、S、X及SIX冲突。

3) 多粒度加锁协议的相容矩阵

加锁申请	数据对象的锁状态 NL	IS	IX	S	SIX	U	X
IS	Y	Y	Y	Y	Y	Y	N
IX	Y	Y	Y	N	N	N	N
S	Y	Y	N	Y	N	Y	N
SIX	Y	Y	N	N	N	N	N
U	Y	Y	N	Y	N	N	N
X	Y	N	N	N	N	N	N

多粒度锁相容矩阵

4) 多粒度加锁/解锁顺序

① 要对一个数据对象加锁，必须对这个数据对象的所有祖先加相应的意向锁，即：申请加锁时，应按自上而下（从根到叶）的次序申请，以便及时发现冲突；

② 解锁时，应按自下而上（从叶到根）的次序进行，以免出现锁冲突。

8.5 死锁及其预防、检测与处理

一. 概述

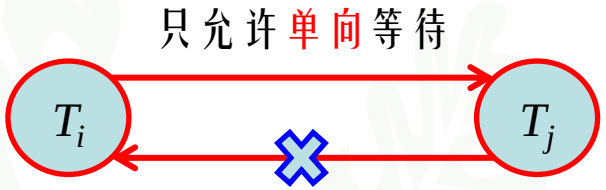
事务间等待关系：一个事务如申请锁而未获准，则需等待其他事务释放锁，从而形成事务间的等待关系。

死锁：当事务出现循环等待时，如不加干预，则会一直等待下去，形成死锁(dead lock)。

对付死锁的二个方法：① 防止死锁；② 检测死锁，发现死锁后处理死锁。

二. 死锁的预防

1. 基本思想



2. 如何实现

3. 基本概念

方法：给每个事务一个优先级，并禁止低优先级的事务等待高优先级事务，或反之。而事务优先级的设定，是当事务开始执行时，给事务一个时间戳TS（Time Stamp）来完成。

时间戳：是一个惟一的、随时间增长的整数。

事务优先级：时间戳越前，事务的优先级越高，也就是说越老的事务，其优先级越高。

示例：如有两个事务T_A和T_B，如$ts(T_A) < ts(T_B)$，则表示T_A早于T_B。

死锁预防策略：① 等待—死亡（Wait-Die）策略；② 击伤—等待（Wound-Wait）策略。

描述策略之前的假定：设T_B已持有某数据对象的锁，现在T_A申请同一数据对象的锁。

4. 等待—死亡（Wait-Die）策略

```
if ts(TA) < ts(TB)
        TA waits;                /*  wait  */
else
{
        rollback  TA;            /* die  */
        restart  TA with the same ts(TA);
}
```

说明：

① 上规则说明，总是年老的事务等待年轻的事务，故不会循环等待，从而避免死锁。

② 因年轻而遭回退的事务，由于重启动时仍用以前的ts，故总会变成年老的事务而等待。这样可保证每个事务最终总会由于时间戳早而成为优先级高的事务，从而能获得它所需要的锁。

③ 一个事务一旦获得它所需的所有锁而不再申请锁时，它就不会有被回退的危险。

5. 击伤—等待（Wound-Wait）策略

```
if ts(T_A)>ts(T_B)
      T_A waits;                /*  wait  */
else
{
      rollback  T_B;            /* wound  */
      restart  T_B with the same ts(T_B);
}
```

说明：此策略中，总是年轻的事务等待年老的事务，因而不会出现死锁。

三. 死锁的检测与处理

1. 死锁的检测

检测死锁的方法：① 超时法； ② 等待图 (Wait-for Graph) 法。

1) 超时法

方法：如果一个事务等待锁的时间太长，超过事先设定的时限，则主观认定其处于循环等待中而撤销之。

说明：

① 发现死锁的时间稍长；

② 可能存在误判；

③ 超时时间设置问题。设置越小，误判机会越大；设置太大，则发现死锁的滞后时间会过长。

2）等待图法

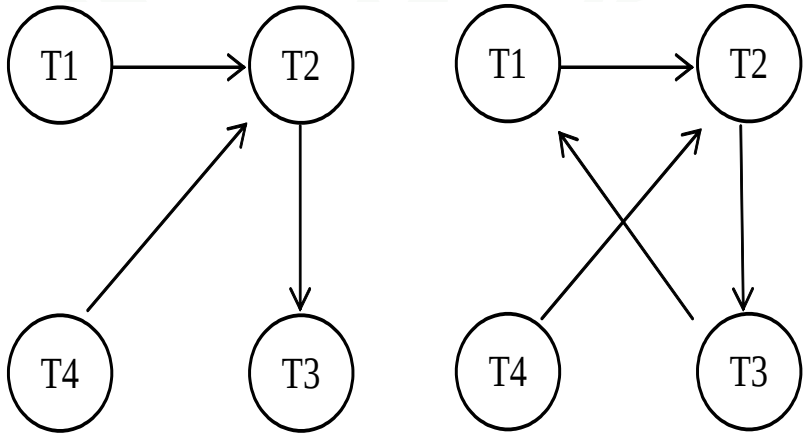
等待图：是一个有向图G=（W，U），顶点W是当前运行事务的集合，U是边的集合，U={(T_i, T_j) | T_i等待T_j, i≠j}。

等待图的维护：当某个锁请求加入队列时，锁管理器会向图中增加一条边；而某个锁请求得到获准时，则它会从图中删除一条相应的边。

死锁检测方法：当且仅当等待图中出现回路时，死锁才发生。

检测改进：当运行事务较多时，维护等待图和检测回路开销较大。如每出现一个等待都要检测一次，虽然可及时发现死锁，但开销太大，影响系统性能。合理的方法是周期性进行死锁检测。

调度时刻	事务 T1	事务 T2	事务 T3	事务 T4
t_0	$Lock_S(A)$			
t_1	R(A)			
t_2		$Lock_X(B)$		
t_3		W(B)		
t_4	$Lock_S(B)$			
t_5			$Lock_S(C)$	
t_6			R(C)	
t_7		$Lock_X(C)$		
t_8				$Lock_X(B)$
t_9			$Lock_X(A)$	



（a）　（b）

t_9 前后的等待图

2. 死锁的处理

处理思想：“杀死”事务，打破循环等待，解除死锁。

死锁处理的步骤：

① 在循环等待的事务中，选一事务作为牺牲者；

② 回退牺牲的事务，释放其获得的锁及其他资源；

③ 将释放的锁让给等待它的事务。

牺牲事务的选择标准：

① 选择最迟交付的事务作为牺牲者；

② 选择获得锁最少的事务作为牺牲者；

③ 选择回退代价最小的事务作为牺牲者。

被“杀死”事务的两种处理方法：

① 告之用户其事务因死锁被杀死，稍后向系统再次交付该事务；

②由DBMS重启动该事务。

说明：不管哪种“挽救”方法，被杀死的事务应等待一段时间才能交付系统，否则，可能再次引起死锁。

8.6 SQL Server中的锁与加锁

SQL Server的加锁：加锁由SQL Server自动处理，一般是以数据页为单位加锁。

锁粒度：整个库、表级、页级、行级和键级。

锁方式：① 共享锁(S)；② 独占锁(X)；③ 更新锁(U)；④ 意向共享锁(IS)；⑤意向独占锁(IX)；⑥共享意向排他锁(SIX)；⑦大容量更新锁(BU)。

相容矩阵：

加锁申请	数据对象的锁状态 NL	IS	IX	S	SIX	U	X
IS	Y	Y	Y	Y	Y	Y	N
IX	Y	Y	Y	N	N	N	N
S	Y	Y	N	Y	N	Y	N
SIX	Y	Y	N	N	N	N	N
U	Y	Y	N	Y	N	N	N
X	Y	N	N	N	N	N	N

SQL Server 多粒度锁相容矩阵

SQL Server中提供的四种事务隔离等级

① “READ UNCOMMITTED”（读未提交的数据）；

② “READ COMMITTED”（读提交数据，或称读交付）：默认；

③ “READ REPEATABLE”（可重复读）；

④ “SERIALIZABLE”（可串行化），同前一项。

SQL SERVER中的这些事务隔离级，可以通过如下形式设置：

```
SET TRANSACTION ISOLATION LEVEL READ COMMITTED
```

事务隔离等级与锁保持的时间关系

SQL Server中，一个对象加共享锁后，应保持到何时释放，这取决于正使用的事务隔离等级，下表描述了它们之间的关系。

事务隔离等级	锁类型	
	共享锁	独占锁
Read Uncommitted	读数据不加S锁	保持到EOT释放
Read Committed	保持到读操作完成后立即释放	保持到EOT释放
Read Repeatable和Serializable	保持到EOT释放	保持到EOT释放

带HOLDLOCK的SELECT：该选项能保证在SELECT语句完成前，或一个含多条SQL语句的事务回退或提交前一直对指定的一张或多张表保持有共享锁。带有HOLDLOCK的SELECT不允许其它进程对其所操作的表加排他锁，可用于在低隔离等级达到重复读效果。

```
BEGIN TRANSACTION
    DECLARE @avg_adv money
    SELECT @avg_adv=avg(advance) FROM titles
    HOLDLOCK
    WHERE type="business"
    IF @avg_adv > 5000
        SELECT title FROM titles WHERE type="business" AND advance
> @avg_adv
  COMMIT TRANSACTION
```

SQL Server的死锁检测与处理：SQL Server自动检测死锁；一旦有死锁发生，则将其中一个导致死锁较次的事务杀死。

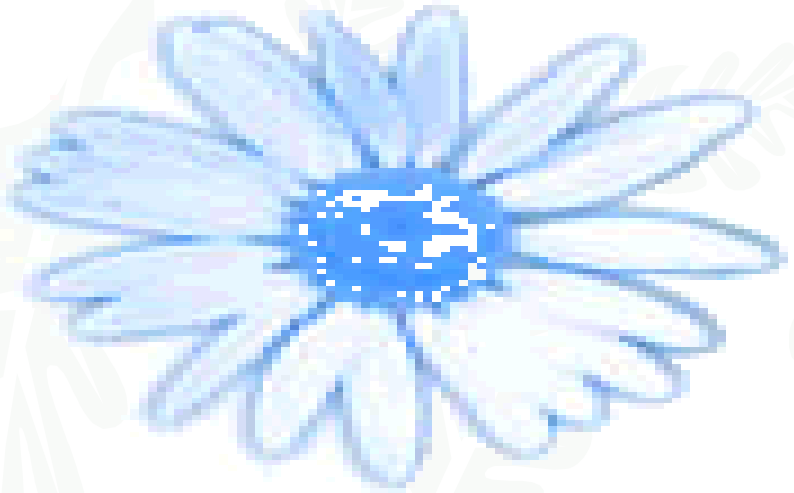
为防止死锁，编程时应注意如下事项：

① 尽量使事务简洁，花时少；

② 少用HOLDLOCK，特别是当事务中需要与用户交互时最好不用HOLDLOCK；

③ 避免同时多于一个用户对同一数据页互操作；

④ 两个事务对两个数据页的访问顺序一致。



—— The End ——